Introduction

In the realm of distributed systems and cloud computing, effective communication and management of remote entities are crucial. This lab focuses on developing a comprehensive system for rover operations using modern web technologies, specifically FastAPI and containerization techniques. Building upon the previous labs' rover and demining scenarios, this project introduces a sophisticated RESTful communication framework between an operator and a server.

The primary objective of this lab is to create a flexible and scalable system that simulates rover management through a web-based interface. By leveraging FastAPI, we will implement a robust HTTP server with multiple endpoints that enable comprehensive rover and map management. The system allows operators to perform critical actions such as:

- Creating, modifying, and deleting rovers
- Managing map configurations
- Tracking mine locations
- Dispatching rovers with specific command sets
- Potentially implementing real-time rover control through WebSocket communication

Moreover, the lab introduces containerization concepts by requiring students to package their application using Docker and deploy it on a cloud platform (Azure). This approach not only demonstrates modern software deployment practices but also provides hands-on experience with cloud infrastructure and container technologies.

The implementation will be divided into several key components:

- A FastAPI server with comprehensive RESTful endpoints
- An operator interface for interacting with the server
- Optional real-time rover control using WebSocket
- Containerization and cloud deployment of the entire system

Through this lab, we gained practical experience in web API design, system communication protocols, cloud deployment, and modern software architecture principles.

Results

Part 1 & 2: The Server an the Operator

GET /map (retrieves the current map, in this case it's the map1.txt file)

```
PS C:\Users\charl\OneDrive\Documents\COE892\lab4works> python RoverOperator.py

==== Rover Operator Menu ====
1. Map Operations
2. Mine Operations
3. Rover Operations
4. Exit
Enter your choice (1-4): 1

==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 1
Map:
0 M 0
0 0 0
M 0 0
0 0 0
```

PUT /map (to update the height and width of the map)

```
==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 2
Enter map height: 5
Enter map width: 5
Map updated successfully

==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 1
Map:
0 M 0 0 0
0 0 0 0 0
M 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

GET /mines (retrieves the list of all the mines, it shows their ID, position, and serial number)

```
==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 3

==== Rover Operator Menu ====
1. Map Operations
2. Mine Operations
3. Rover Operations
4. Exit
Enter your choice (1-4): 2

==== Mine Operations ====
1. List All Mines
2. Get Mine Details
3. Create Mine
4. Update Mine
5. Delete Mine
6. Back to Main Menu
Enter your choice (1-6): 1
Mines:
ID: 1, Position: (1, 0), Serial: b1l3qy2l9g
ID: 2, Position: (0, 2), Serial: tapsgyjqd1
```

GET /mines/id: (retrieves the details of a mine given an ID, in this case if ID 1 is given it shows the ID, position, and serial number of mine 1)

```
==== Mine Operations ====
1. List All Mines
2. Get Mine Details
3. Create Mine
4. Update Mine
5. Delete Mine
6. Back to Main Menu
Enter your choice (1-6): 2
Enter mine ID: 1
Mine ID: 1
Position: (1, 0)
Serial Number: b1l3qy2l9g
```

POST /mines (creates a mine, figure on the right shows the updated map with the new mine)

```
==== Mine Operations ====
1. List All Mines
2. Get Mine Details
3. Create Mine
4. Update Mine
5. Delete Mine
6. Back to Main Menu
Enter your choice (1-6): 3
Enter X coordinate: 3
Enter Y coordinate: 3
Enter serial number: mjh2vzflp7
Mine created successfully with ID: 3
```

```
==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 1
Map:
0 M 0 0 0
0 0 0 0 0
M 0 0 0 0
0 0 0 M 0
0 0 0 0 0
```

DELETE /mines/id: (deletes a mine, figure on the right shows the updated map with the deleted mine)

```
==== Mine Operations ====
1. List All Mines
2. Get Mine Details
3. Create Mine
4. Update Mine
5. Delete Mine
6. Back to Main Menu
Enter your choice (1-6): 5
Enter mine ID to delete: 3
Mine 3 deleted successfully
```

```
==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 1
Map:
0 M 0 0 0
0 0 0 0 0
M 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

PUT /mines/:id (updates a mine, given a mine ID, it can update the position and serial number of the mine, can also leave blank if don't want to change)

```
==== Mine Operations ====
1. List All Mines
2. Get Mine Details
3. Create Mine
4. Update Mine
5. Delete Mine
6. Back to Main Menu
Enter your choice (1-6): 4
Enter mine ID: 2
Enter new X coordinate (leave blank to keep current): 3
Enter new Y coordinate (leave blank to keep current): 3
Enter new serial number (leave blank to keep current):
Mine 2 updated successfully
```

```
==== Map Operations ====
1. Get Map
2. Update Map
3. Back to Main Menu
Enter your choice (1-3): 1
Map:
0 M 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 M 0
0 0 0 0 0
```

POST /rovers (create rovers)
When Rover 1 is dispatched it should explode on Mine 2.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 3
Enter commands (e.g., LMRMMD): MMMLMMMM
Rover created successfully with ID: 1
```

When Rover 2 is dispatched it should dig Mine 1

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 3
Enter commands (e.g., LMRMMD): LMDMMM
Rover created successfully with ID: 2
```

Rover 3 is created for deletion later

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 3
Enter commands (e.g., LMRMMD): MLMRMDM
Rover created successfully with ID: 3
```

GET /rovers (list of all the rovers)

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 1
Rovers:
ID: 1, Status: Not Started
ID: 2, Status: Not Started
ID: 3, Status: Not Started
```

GET /rovers/id:

(retrieves the status, current position, and commands of the rover given by their id)

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 2
Enter rover ID: 1
Rover ID: 1
Status: Not Started
Position: (0, 0)
Commands: MMMLMMMM
```

DELETE /rovers/id: (delete rovers)
Deleted Rover 3 as stated earlier. Figure on the right shows that there are only 2 rovers, Rover 1 and 2.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 5
Enter rover ID to delete: 3
Rover 3 deleted successfully
```

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 1
Rovers:
ID: 1, Status: Not Started
ID: 2, Status: Not Started
```

POST /rovers/id:/dispatch (dispatch a rover)
Figure on the left shows that Rover 1 was dispatched. Figure on the right shows that Rover 2 was dispatched.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 6
Enter rover ID to dispatch: 1
Rover 1 dispatched successfully
Rover dispatched. Final position: (3, 3)
Executed commands: MMMLMMM
```

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 6
Enter rover ID to dispatch: 2
Rover 2 dispatched successfully
Rover dispatched. Final position: (4, 0)
Executed commands: LMDMMM
```

Just as expected earlier, Rover 1's status is "Eliminated" because it did not disarm a mine and therefore exploded. While Rover 2's status is "Finished" signalling that it has finished all of its moves without exploding.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 1
Rovers:
ID: 1, Status: Eliminated
ID: 2, Status: Finished
```

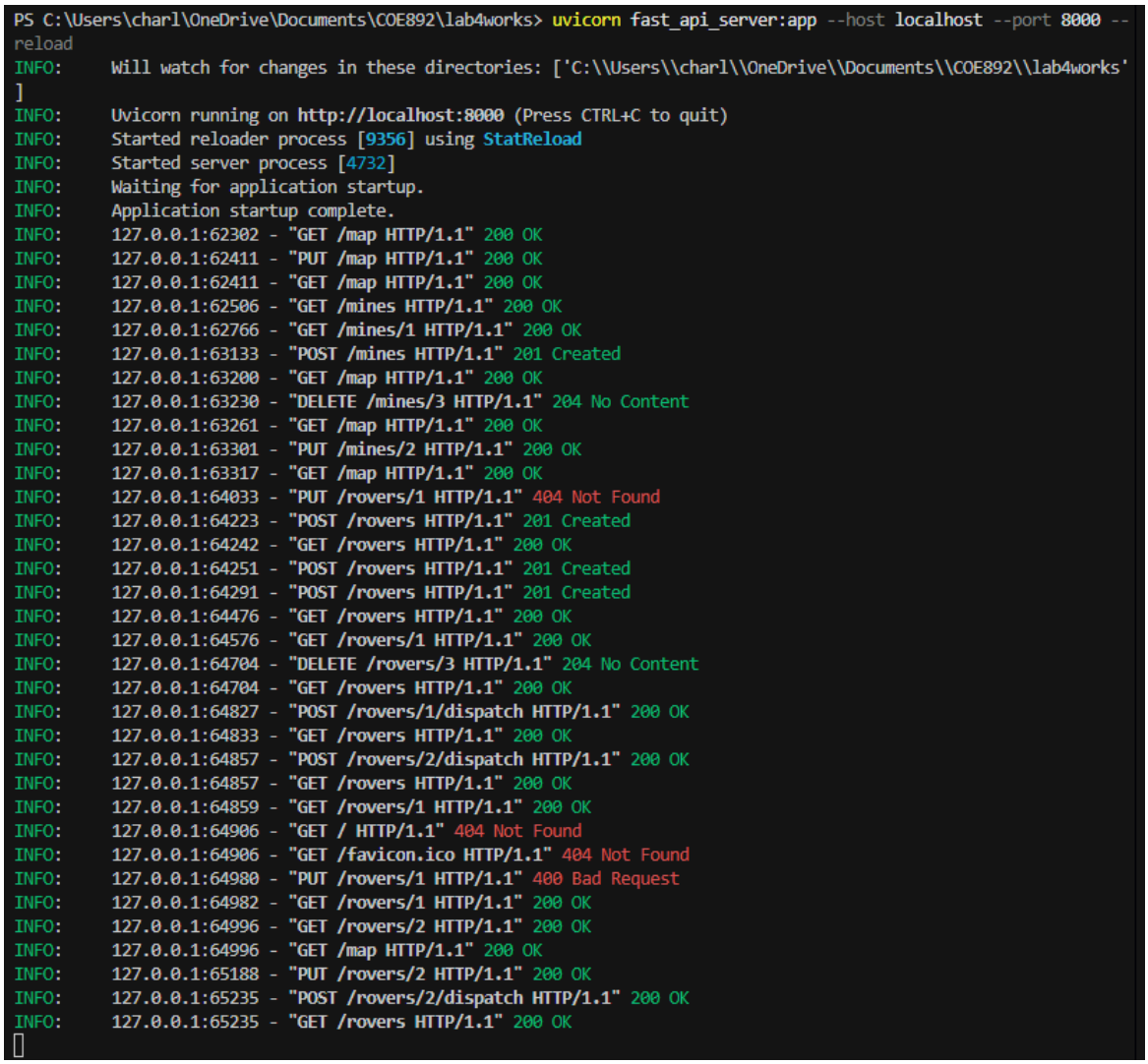
PUT /rovers/id: (updates the new commands for the rover)

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 4
Enter rover ID: 2
Enter new commands (e.g., LMRMMD): RMMMMM
Rover 2 updated successfully
```

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 6
Enter rover ID to dispatch: 2
Rover 2 dispatched successfully
Rover dispatched. Final position: (0, 0)
Executed commands: RMMMMM
```

Server

Just a screenshot of what the server looks like while operator was running



```
PS C:\Users\charl\OneDrive\Documents\COE892\lab4works> uvicorn fast_api_server:app --host localhost --port 8000 --
reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\charl\\OneDrive\\Documents\\COE892\\lab4works'
]
INFO:     Uvicorn running on http://localhost:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [9356] using StatReload
INFO:     Started server process [4732]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:62302 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:62411 - "PUT /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:62411 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:62506 - "GET /mines HTTP/1.1" 200 OK
INFO:     127.0.0.1:62766 - "GET /mines/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:63133 - "POST /mines HTTP/1.1" 201 Created
INFO:     127.0.0.1:63200 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:63230 - "DELETE /mines/3 HTTP/1.1" 204 No Content
INFO:     127.0.0.1:63261 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:63301 - "PUT /mines/2 HTTP/1.1" 200 OK
INFO:     127.0.0.1:63317 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:64033 - "PUT /rovers/1 HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:64223 - "POST /rovers HTTP/1.1" 201 Created
INFO:     127.0.0.1:64242 - "GET /rovers HTTP/1.1" 200 OK
INFO:     127.0.0.1:64251 - "POST /rovers HTTP/1.1" 201 Created
INFO:     127.0.0.1:64291 - "POST /rovers HTTP/1.1" 201 Created
INFO:     127.0.0.1:64476 - "GET /rovers HTTP/1.1" 200 OK
INFO:     127.0.0.1:64576 - "GET /rovers/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:64704 - "DELETE /rovers/3 HTTP/1.1" 204 No Content
INFO:     127.0.0.1:64704 - "GET /rovers HTTP/1.1" 200 OK
INFO:     127.0.0.1:64827 - "POST /rovers/1/dispatch HTTP/1.1" 200 OK
INFO:     127.0.0.1:64833 - "GET /rovers HTTP/1.1" 200 OK
INFO:     127.0.0.1:64857 - "POST /rovers/2/dispatch HTTP/1.1" 200 OK
INFO:     127.0.0.1:64857 - "GET /rovers HTTP/1.1" 200 OK
INFO:     127.0.0.1:64859 - "GET /rovers/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:64906 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:64906 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:64980 - "PUT /rovers/1 HTTP/1.1" 400 Bad Request
INFO:     127.0.0.1:64982 - "GET /rovers/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:64996 - "GET /rovers/2 HTTP/1.1" 200 OK
INFO:     127.0.0.1:64996 - "GET /map HTTP/1.1" 200 OK
INFO:     127.0.0.1:65188 - "PUT /rovers/2 HTTP/1.1" 200 OK
INFO:     127.0.0.1:65235 - "POST /rovers/2/dispatch HTTP/1.1" 200 OK
INFO:     127.0.0.1:65235 - "GET /rovers HTTP/1.1" 200 OK
```

Part 3: The Websocket (control the rover in real-time)
Here is an example of when I controlled the rover and made it step on a mine for it to explode.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 8
Enter rover ID for real-time control: 1
Connected to rover 1 for real-time control
Commands: L (turn left), R (turn right), M (move forward), D (dig)
Enter 'q' to quit
Map:
R M 0
0 0 0
M 0 0
0 0 0
```

```
Enter command (L/R/M/D/q): M
Command M executed successfully
New position: (0, 1)
Map:
0 M 0
R 0 0
M 0 0
M 0 0
0 0 0

Enter command (L/R/M/D/q): M
Enter command (L/R/M/D/q): M
WARNING: Rover is on a mine! Disarm it before moving.
WARNING: Rover is on a mine! Disarm it before moving.
Map:
0 M 0
0 M 0
0 0 0
R 0 0
0 0 0

Enter command (L/R/M/D/q): M
ROVER ELIMINATED: Rover eliminated! Stepped on a mine without disarming
```

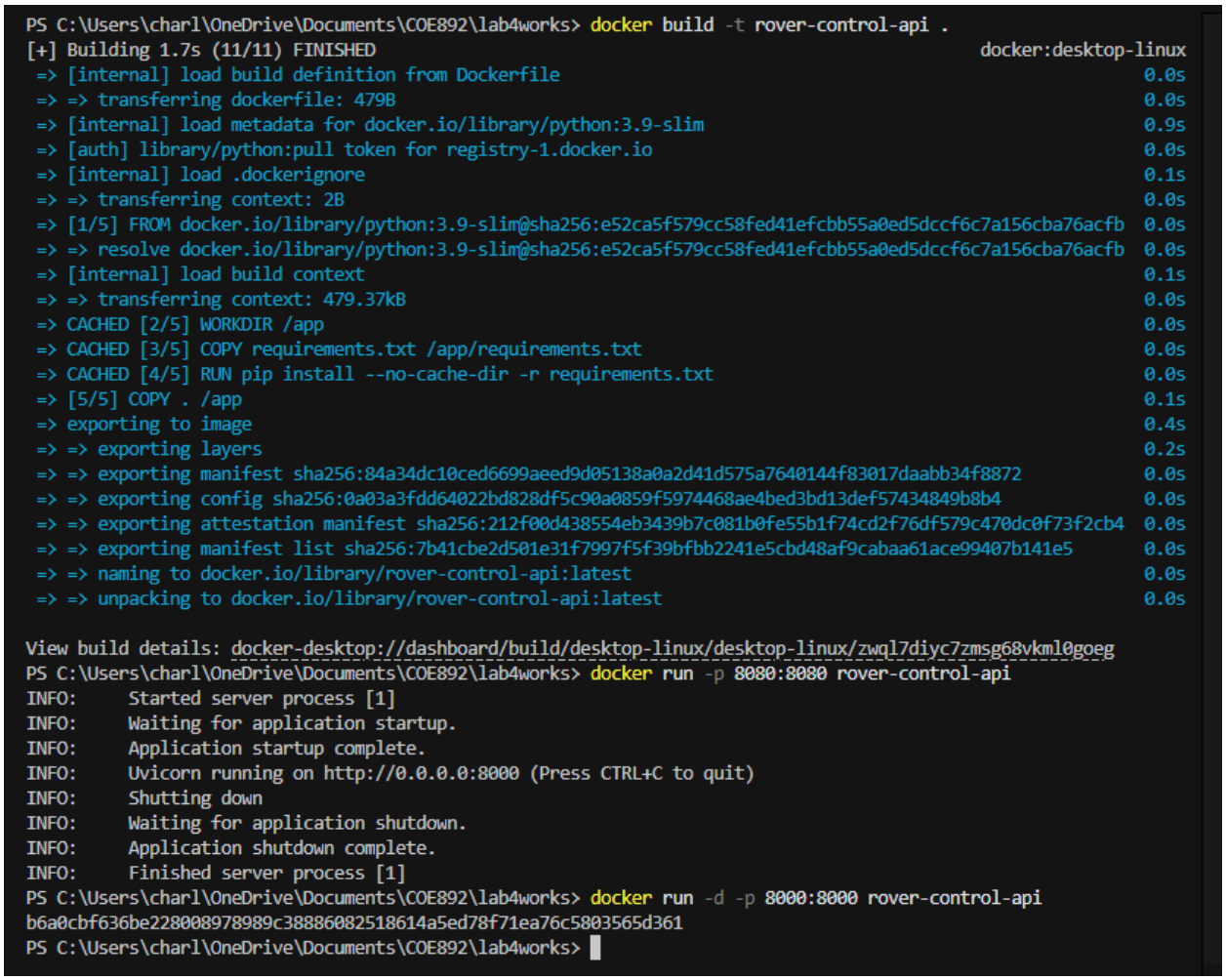
Here is another example, where I controlled a rover but this time I disarm a mine and the rover didn't explode.

```
==== Rover Operations ====
1. List All Rovers
2. Get Rover Details
3. Create Rover
4. Update Rover
5. Delete Rover
6. Dispatch Rover
7. Display Rover Path
8. Control Rover (Real-time)
9. Back to Main Menu
Enter your choice (1-9): 8
Enter rover ID for real-time control: 1
Connected to rover 1 for real-time control
Commands: L (turn left), R (turn right), M (move forward), D (dig)
Enter 'q' to quit
Map:
R M 0
0 0 0
M 0 0
0 0 0

Enter command (L/R/M/D/q): M
Command M executed successfully
New position: (0, 1)
Map:
0 M 0
R 0 0
M 0 0
0 0 0
```

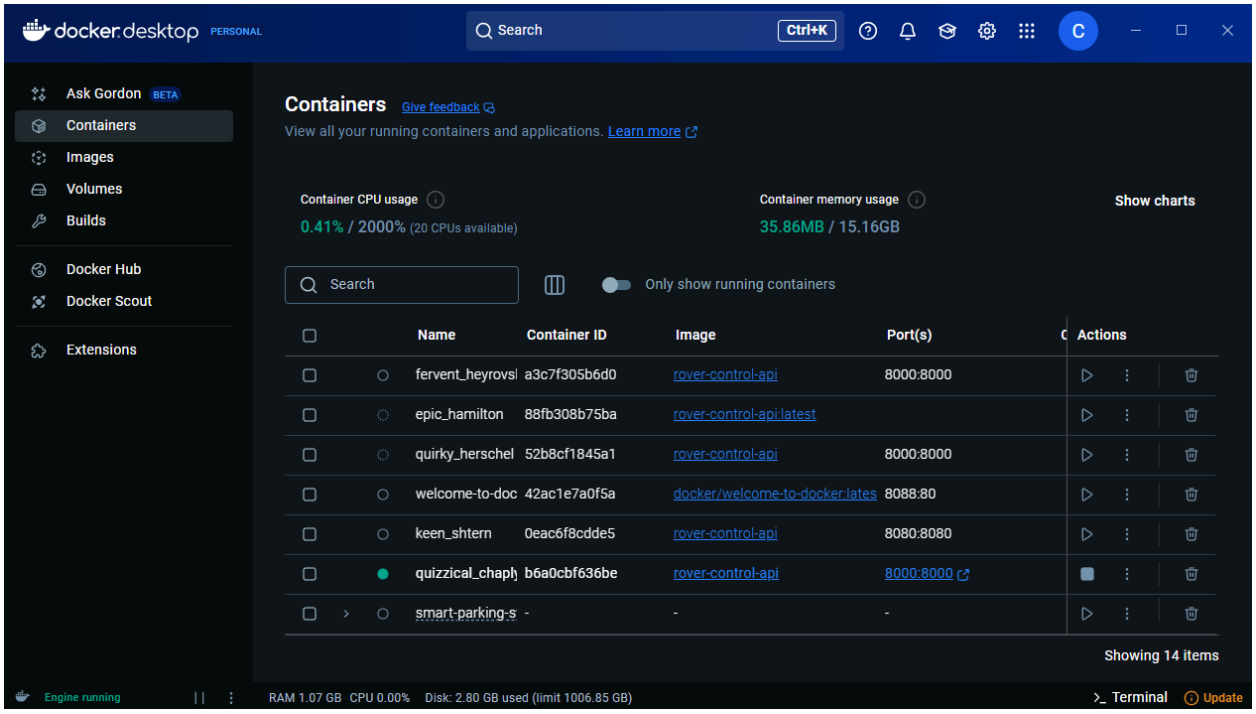
```
Enter command (L/R/M/D/q): M
WARNING: Rover is on a mine! Disarm it before moving.
Map:
0 M 0
0 0 0
R 0 0
0 0 0

Enter command (L/R/M/D/q): D
Command D executed successfully
Mine disarmed! PIN: 100000
Map:
0 M 0
0 0 0
R 0 0
0 0 0
```

Part 4: Azure

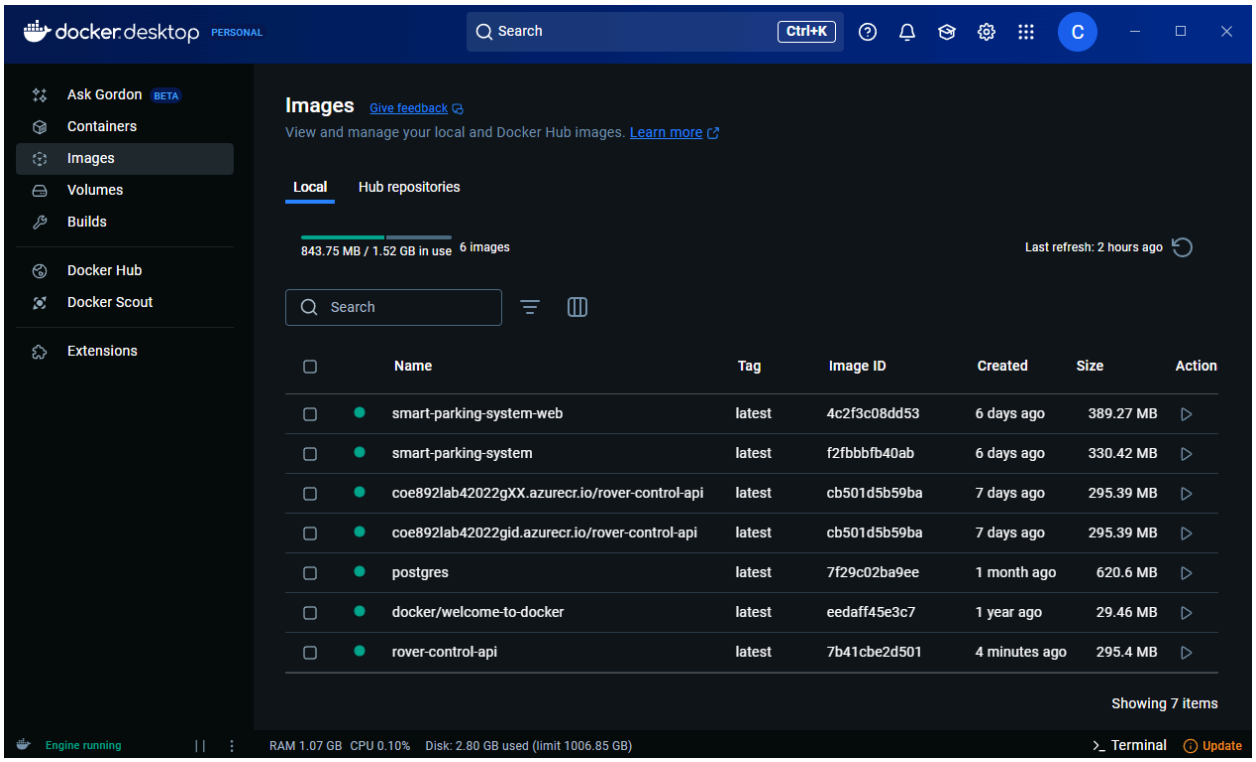
Screenshot of my terminal building a container in docker and running it.



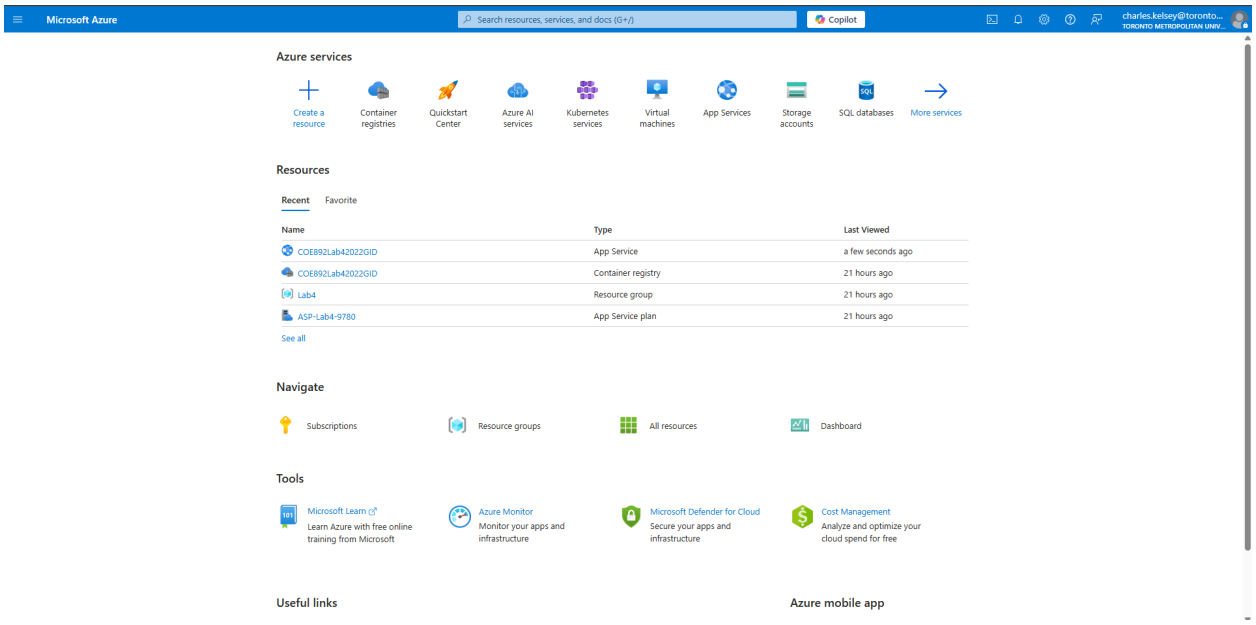
Screenshot of the containers in my docker desktop.



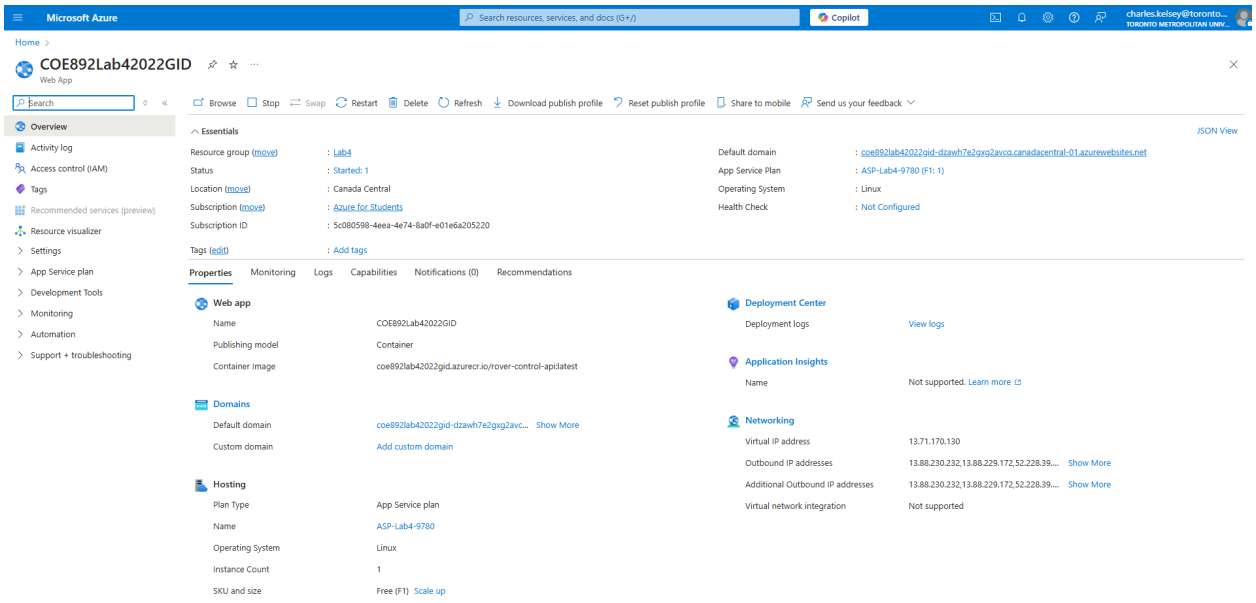
Screenshot of the images in my docker desktop.



Screenshots in Azure that I have a Docker Repository and deployed my container to the remote container registry.



Screenshot of the web app I built using the container image.



Conclusion

This lab project represented a comprehensive exploration of modern web technologies, distributed system design, and cloud deployment strategies. By implementing a rover management system using FastAPI, we gained deep insights into several critical aspects of contemporary software development and system architecture.

The project successfully demonstrated the power of RESTful API design through carefully crafted endpoints that enable comprehensive rover and map management. By implementing methods for creating, updating, dispatching, and tracking rovers and mines, we developed a flexible system that mimics real-world remote operation scenarios.

Key technical achievements of this lab included:

- Designing a robust FastAPI server with multiple endpoints
- Implementing complex routing and request handling
- Creating a flexible data management system for rovers and map configurations
- Exploring WebSocket technology for potential real-time communication
- Containerizing the application using Docker
- Deploying the application on a cloud platform (Azure)

The lab also highlighted the importance of modular system design, where different components can interact seamlessly through well-defined interfaces. The RESTful approach allowed for clear separation of concerns, making the system both extensible and maintainable. Containerization and cloud deployment components introduced practical skills in modern software infrastructure. By packaging the application in a Docker container and deploying it to Azure, we gained hands-on experience with cloud technologies that are increasingly crucial in contemporary software development.

The optional WebSocket implementation further expanded our understanding of real-time communication protocols, demonstrating how advanced web technologies can enable dynamic, interactive systems.

Beyond technical skills, this lab emphasized the importance of:

- Comprehensive system design
- Handling various edge cases and potential failure scenarios
- Creating user-friendly interfaces
- Implementing scalable and flexible software architectures

As distributed and cloud computing continue to evolve, the skills and insights gained from this lab provide a solid foundation for developing complex, interconnected software systems.